

Tennis genetic algorithm

Group - Iga Świątek Fanclub

Magdalena Rapała
Miłosz Liniewiecki

AEI faculty

24.06.2026

Agenda

- 1 Introduction to problem and environment
- 2 Flowchart
- 3 Pseudocode
- 4 Results

Introduction to problem and environment

Tennis is an Atari Gymnasium environment with two players.

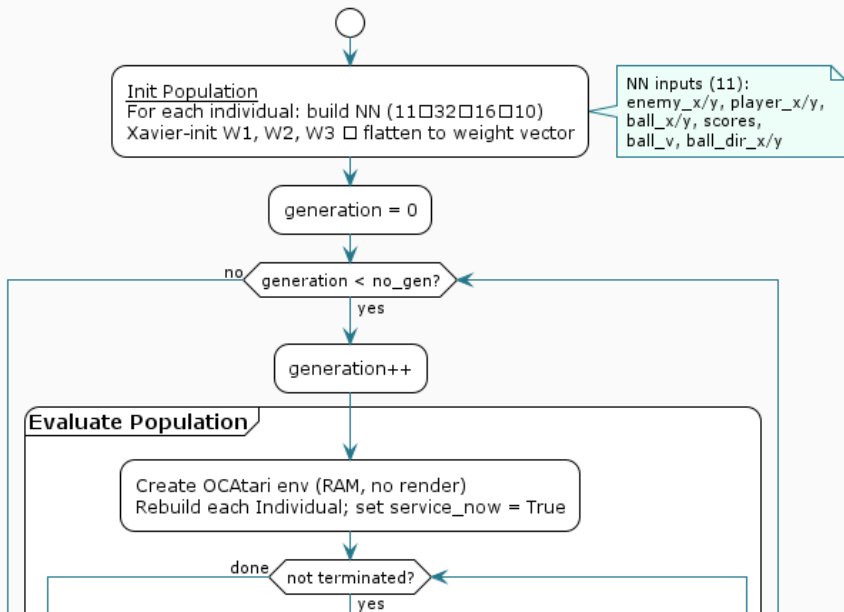
- Blue player - Game AI, our enemy.
- Orange player - Our AI, the one we trained.

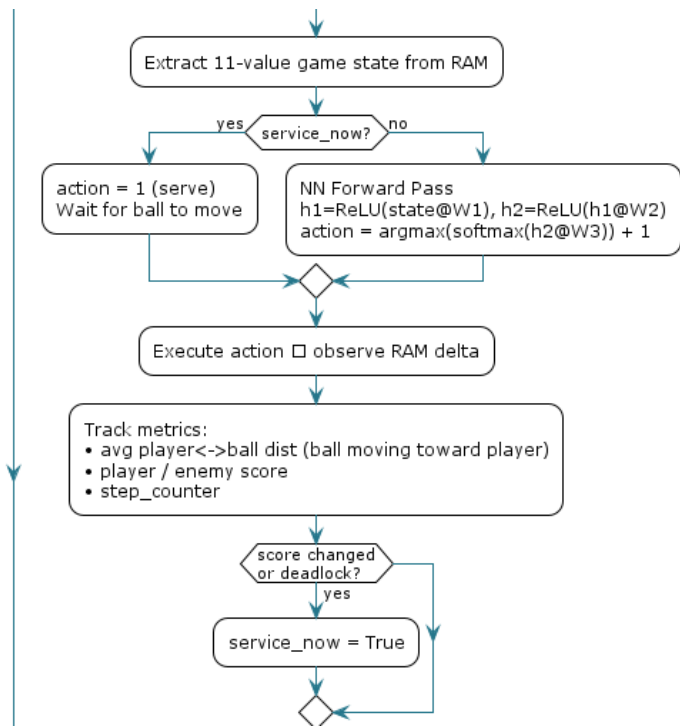
Atari Gymnasium does not give you any data to feed into the genetic algorithm, which is why we used OCArari for RAM values extraction.



Figure: Tennis environment during training.

Tennis Genetic Algorithm Flowchart





time $\geq$ max_time
or steps $\geq$ max_steps?

yes

Break early

Compute progressive fitness f:
f = (1 - avg_dist)
gen > 10 $\square$ f += frames_survived
gen > 20 $\square$ f += (1 - missed_dist)
gen > 30 $\square$ f += win_fraction

On Generation

Print best fitness

generation % 5 == 0?

yes

Save best weights to /tmp/best_fits/

Selection ☐ **Crossover** ☐ **Mutation**

Rank by fitness
Keep top-1 elite (elitism)
SSS: select parents_mating parents

Uniform crossover:
each gene inherited from parent A or B

Random mutation:
gene += uniform(-0.15, 0.15)
for mutation_percent_genes % of genes

Replace population with new offspring

Return best individual
(highest fitness weight vector)

Pseudocode 1/6 - Init

```
FUNCTION init_population(no_inv):  
  population <- []  
  
  FOR i = 1 TO no_inv DO  
    Build NN with layers: n_x=11 -> h1=32 -> h2=16 -> n_y=10  
    Xavier-init weight matrices W1, W2, W3  
    population.append(flatten([W1, W2, W3]))  
  END FOR  
  
  RETURN population
```


Pseudocode 2/6 - Individual init

```
population <- init_population(no_inv)
generation <- 0

WHILE generation < no_gen DO

    generation <- generation + 1

    FOR each individual in population DO

        Create OCArari "Tennis-v4" env (RAM mode, no render)
        Rebuild Individual from weight vector
        service_now <- True
```

Pseudocode 3/6 - State extraction

```
WHILE not terminated DO
  state <- extract 11-value game state from RAM
  IF service_now THEN
    action <- 1 (force serve)
    IF ball started moving THEN
      service_now <- False
    END IF
  ELSE
    h1 <- ReLU(state @ W1)
    h2 <- ReLU(h1 @ W2)
    logits <- h2 @ W3
    action <- argmax(softmax(logits)) + 1
  END IF
  Execute action in env -> observe RAM delta
```

Pseudocode 4/6 - Safety checks

Track metrics:

```
    avg_player_ball_dist  <- distance(player, ball)
        when ball moves toward player
    player_score, enemy_score
    step_counter <- step_counter + 1
```

```
IF score changed OR deadlock detected THEN
    service_now <- True
END IF
```

```
IF elapsed_time >= max_time
    OR step_counter >= max_steps THEN
    BREAK (early termination)
END IF
```

```
END WHILE
```

Pseudocode 5/6 - Fitness calculation

Compute fitness f:

$f \leftarrow 1 - \text{avg_player_ball_dist}$

IF generation > 10 THEN $f \leftarrow f + (\text{step_counter} / \text{max_steps})$

IF generation > 20 THEN $f \leftarrow f + (1 - \text{avg_missed_distance})$

IF generation > 30 THEN $f \leftarrow f + (\text{player_score} / \text{total_points})$

Store f as fitness of this individual

END FOR

Pseudocode 6/6 - Selection, best result

Print best fitness of current generation

IF generation MOD 5 = 0 THEN

 Save best weight vector to /tmp/best_fits/gen-{g}-fit-{f}.npy

END IF

Rank population by fitness

elite <- top-1 individual (preserved unchanged)

parents_mating <- floor(no_inv x parents_frac)

parents <- SSS_select(population, parents_mating)

new_population <- [elite]

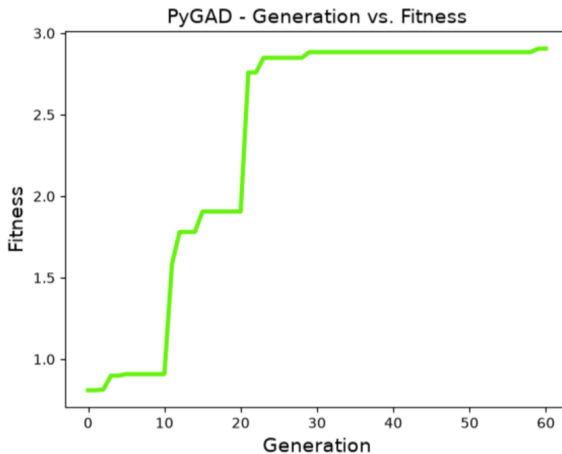


Figure: Results